Jev in a Multi-Step Workflow: Predicting Failures Is Not Knowing Which Reviews Pay

Thejas P Rao
thejasrao262003@gmail.com

Tushar M
tusharm3040@gmail.com

Dr Prajwala T R
PES University, Bengaluru, India
prajwalatr@pes.edu

Abstract

Many systems chain an AI model's typed decisions into a workflow and use the model's reported confidence to decide which cases a human should check. We test how well that works, using Jev, TypeSafe's typed decision model and its three primitives (a categorical `choice`, a binary `noul` and an ordinal `score`), on a bug-triage workflow. We keep two things apart that are usually run together: whether a signal can tell which chained decisions are wrong, and whether it can tell which of those are worth a human's time. We study 442 Mozilla and Linux kernel bugs, then run pre-registered tests on 500 new bugs, four further models and an independent Kubernetes workflow. Failure is disagreement with a target action we construct from reference labels. A frozen decision-feature estimator ranks failures better than final-step confidence (AUROC 0.76 vs. 0.55 on the new bugs, supported for all five models), and its signal is almost entirely the historical error rate of the chosen action. The Kubernetes replication, on a chronological split, confirmed this (AUROC 0.78 vs. 0.63), as did three of four further models, including the only other one competent at every step. These error rates do not transfer across label mixes: fitted on a quota-sampled development set, they can rank natural-mix errors below chance. Re-weighting to the deployment mix repairs the ranking, but the label-free estimate of that mix is biased and made ranking worse on Kubernetes. Prediction did not become review value: under the pre-registered costs no 10% review policy beat no review, an oracle locates the value in wrongly accepted cases the tested signals do not isolate, and whether review pays depends on the cost ratio. Being able to flag the likely-wrong decisions did not, on its own, give us a review policy that saved money.

1 Introduction

A model can return a well-typed, valid-looking answer and still be wrong. When several such answers are chained into a workflow, a mistake at one step feeds into the next. A common assumption is that the confidence each step reports can be combined into a single, workflow-level signal that says when to hand a case to a human. We check whether that assumption holds up.

Prior work has benchmarked Jev's primitives and calibration on single tasks [11, 12, 29], and recent work has examined uncertainty across multi-step agent trajectories [26]. We study a different, operational question. *When a workflow chains several typed decisions, which signals identify the cases where it fails, and can those signals identify the specific costly errors for which human review is worthwhile?* The two do not have to go together: a signal can be good at ranking failures and still be no help in choosing which ones to review. This matters because Jev's three primitives report uncertainty in *structurally* different forms: `choice` picks from unordered labels and returns a

probability for each plus a scalar confidence, `score` rates an ordered scale and returns an ordinal distribution plus a scalar confidence, and `noul` answers yes/no and returns a single probability with no separate confidence field. Treating these numbers as one kind of quantity is a modeling assumption, and this paper tests it.

TypeSafe's documentation describes Jev's `confidence` for `choice` and `score` as a number that "collapses" the output distribution into a value between 0 and 1. It is 1 when all probability is on one outcome and 0 when probability is spread evenly. The documentation calls this "one reasonable way to summarize a distribution, not the only one", and advises setting thresholds from one's own data [40]. For `noul`, it returns "a single probability" with "no separate `confidence`". Our results are therefore not a test of a calibration claim. They measure how these documented concentration statistics behave as escalation signals when Jev's primitives are chained, which the documentation leaves open.

Failure here means disagreement between the workflow's action and a target action we construct from reference labels (Section 3); we return to targets that do not depend on that construction in Section 9. We ask three research questions. The first (RQ1) is which signals rank workflow failures: the models' confidence, or what the workflow decided. The second (RQ2) is whether those signals stay stable when the label distribution changes between development and deployment. The third (RQ3) is whether signals that rank failures also identify the reviews that reduce cost.

What is new here is mostly in how the evaluation is set up. We measure uncertainty at the level of a whole *workflow* of structurally different typed decisions joined by an explicit action rule, so a failure is defined on the action the workflow takes, not on any single answer, and every step-level signal can then be judged as a predictor of that failure. We also keep apart two questions that usually get run together (whether a signal ranks failures, and whether acting on it actually pays), by scoring a fixed review budget against a frozen cost matrix, random review and an oracle bound. And we try to keep the evaluation itself honest: every confirmatory test was pre-registered and frozen by hash before its data existed, and the main finding was replicated across five models and on a separate, chronologically split workflow. Section 3.6 gives the details.

For RQ1, a frozen decision-feature estimator ranks failures better than final-step confidence on 500 new bugs, and the gap replicates across five models from four sources and in an independent Kubernetes workflow (Sections 8 to 10). Paired ablations show that the signal is almost entirely the historical error rate of the chosen action; for the two Jev-style models the estimator adds nothing detectable to that rate, and confidence adds little except for the weakest general models. For RQ2, those error rates are not portable across label mixes: fitted on a quota-sampled development set, they can rank a natural-mix set's errors below chance on targets that bypass our construction. Re-weighting to the deployment label mix repairs the ranking, even with weights estimated from the models' own predictions without test labels when those predictions discriminate the label, but under known synthetic shifts that label-free estimate is biased toward the development mix, and it made ranking worse on Kubernetes. For RQ3, under the pre-registered costs no 10% review policy beat no review (Section 11). An oracle locates the value in wrongly accepted cases that the tested signals do not isolate, an exploratory sensitivity analysis shows that whether review pays depends on the ratio of wrong-accept cost to review cost, and a case-level score failed a pre-registered feasibility check, so its confirmatory study was stopped before new data. Supporting results show that the primitives' confidence signals behave differently inside the workflow, that fixed thresholds set on constructed content do not transfer, and that `score` compresses severity on two trackers, consistent with single-task audits of Jev [12, 29] (Sections 4 to 7); the study's own audit process caught seven defects, five of them before the data that would have exposed them existed (Section 12).

We do not claim anything about general calibration, about other Jev versions or other APIs, about domains beyond software issue triage, or about real-world operational benefit. The remaining

sections cover related work (Section 2) and the methodology (Section 3), then the exploratory analyses on the original trackers (Sections 4 to 7), the pre-registered prediction tests (Sections 8 to 10), and the review-value test (Section 11), followed by the audits and threats to validity (Sections 12 and 13) and the conclusion (Sections 14 and 15).

2 Related work

Evaluations of Jev and open decision models. Jev has been benchmarked zero-shot on 37 datasets (346,009 requests), where its `choice` probabilities were well calibrated and supported selective prediction, while its binary probabilities ranked well but sat poorly relative to a 0.5 threshold, and thresholds tuned on training data improved F1 [12]. A calibration audit with released per-item outputs (described by its author as exploratory) compared Jev with open and general-purpose classifiers on SST-2 and AG News [29]. It found that calibration changed with the question form (a yes/no form was underconfident, a choice form close to calibrated), that 71.6% of four-class answers sat at exactly 1.00, so a threshold on the top-class probability had only 53 operating points, that one fitted temperature removed most miscalibration, and that no threshold on Jev's scores certified a 5% automatic tier on the four-class task under its procedure; and a comparison on seven political-science replications found its calibration advantage over other models not consistent [11]. An early evidence audit of 28 papers on typed decision models found no independent accuracy advantage yet for the typed readout over comparable label-probability readouts, and proposed an evaluation checklist [38]. Open instruction-tuned models can produce Jev-like decisions by reading answer-label logits in one forward pass [19], the same kind of readout we use for the general models in Section 9. These studies evaluate single requests; none studies `score` inside a workflow, composition of dependent decisions, or the cost of human review. Our results agree with them where they overlap: `choice` confidence is saturated yet discriminative, and thresholds must be validated on the target data, which our transfer result shows directly. On `noul` the measures differ. The audits report calibration and accuracy on sentiment, where the yes/no form was accurate and recalibratable; on our regression question, `noul`'s probability did not discriminate its own correctness (AUROC near or below 0.5). Binary-probability quality therefore looks task-dependent.

Calibration of single decisions. Post-hoc calibration [31, 34] and the miscalibration of modern networks [21] are well studied, and confidence estimation for language models has been surveyed in [18].

Selective prediction and deferral. Risk–coverage trade-offs and reject options [15–17], learning to defer [28, 30] and prediction sets in human–AI teams [5] address *one* decision. In human–AI teaming, the most accurate model need not give the best team: optimizing expected team utility, which accounts for verification cost and both parties' accuracy, beats optimizing accuracy [6]. Closest to our intervention result, recent work argues that error risk is not review value under bounded review budgets: review priority should depend on wrongness, repairability, impact and cost, and ranking by review value left less residual error than ranking by risk on a question-answering benchmark [33]. Our intervention finding is an instance of that distinction in a dependent typed-decision workflow, with a frozen cost table and pre-registered tests.

Label shift. When class priors change between training and deployment, posteriors can be re-adjusted to the new priors [37], and the new priors can be estimated from a black-box predictor's

confusion matrix and its predictions on unlabeled deployment data, provided that matrix is invertible [27]. Our RQ2 finding is a consequence of this for reliability estimates: an error rate conditioned on a decision is itself a function of the label prior, and we use the second method to correct it without test labels.

Reliability of multi-stage software systems. Component-based reliability models treat the probability that an error propagates from one component to the next as an architectural attribute that shapes where to place detection and recovery [10]. Bug severity prediction from report text is a long-studied triage task [25]. We study a related propagation problem for chained model decisions, with the escalation point as the recovery mechanism.

Conformal methods and composition. Conformal prediction [3] and conformal risk control [4] give distribution-free guarantees. Composition across stages has been studied for two stages [46] and for multi-stage NLP and LLM pipelines, where errors compound across stages and independently calibrated stages lack a joint coverage guarantee [24]. Error propagation through pipeline steps has also been quantified [9]. Classical reliability theory composes component reliabilities [7].

Cascades and escalation. Cascades route a *single* query through model tiers. Prior work studies early abstention [43], the optimal depth and deferral thresholds of cascades [8], conformal cascades [13], confidence tuning [36] and judges with escalation guarantees [22].

Combining confidence across components. Fixed combination rules such as the product rule are classical [23]. The product rule's usual justification assumes conditional independence and comparable probability scales, and these are precisely the assumptions our strategy S2 makes implicitly. At the system level, a recent taxonomy argues that uncertainty in LLM applications must be studied across the whole compound system, as it propagates within models, across system components and through socio-technical processes [42]. Cost-sensitive deferral to multiple experts under workload limits [2] is closely related to the budgeted escalation setting our follow-up uses.

Ordinal outputs and central tendency. LLM raters have been reported to compress ordinal scores toward the middle of the scale, for example in clinical scoring, where prompt changes did not remove the effect [44]. Few-label calibration of ordinal LLM outputs models such flattening explicitly [41]. Our severity result is consistent with this literature. What we add is that it appears in bug triage, against two labeling processes, and that the primitive's own confidence gives no sign of it.

Agents and trajectories. Closest to our workflow framing, a counterexample shows that step-level calibration need not imply trajectory-level calibration for LLM agents, and step errors are empirically coupled along trajectories [26]. Related work calibrates agent trajectories [45] and studies uncertainty propagation in multi-step agentic decisions [14].

Positioning. Prior work benchmarks Jev's primitives on single requests, studies calibration of single decisions, cascades of the *same* decision and agent trajectories, and argues in general that risk differs from review value. We do something more specific: we take a dependent workflow of three *structurally different* typed decisions and, against real labels, held-out data and five models, test which signals predict workflow failure and whether they find the costly errors that make human review worthwhile. We did not find prior work with this exact setup. Our literature search was

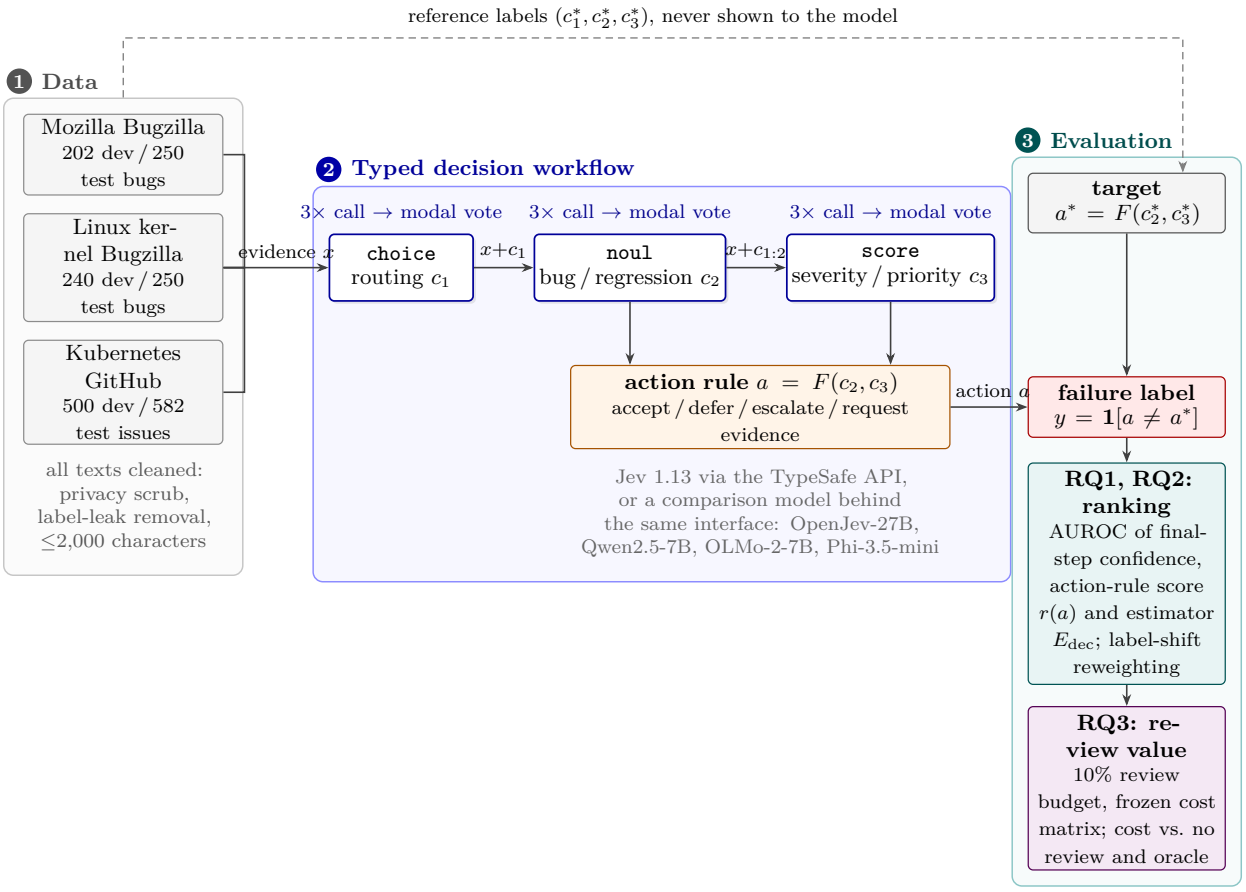


Figure 1: Study architecture. (1) Bug and issue reports from three public trackers are cleaned and stripped of label leaks. (2) The evidence passes through a dependent chain of three typed decisions, each called three times and reduced by modal vote, with upstream decisions appended to the evidence; an explicit action rule F maps the second and third decisions to an action. The same chain runs on Jev and on four comparison models. (3) Reference labels, never shown to the model, give the target action a^* through the same rule; a failure is $a \neq a^*$. Failure signals are evaluated for ranking (RQ1, RQ2) and for review value under a fixed budget and frozen costs (RQ3).

web-based and did not systematically cover reliability engineering or HCI escalation research (we added the closest work on error propagation, human–AI teaming, deferral and label shift in a later pass); most of the Jev papers we cite appeared in September and October 2026.

3 Methodology

3.1 Study architecture

Figure 1 shows the three stages of the study. Bug reports from the Mozilla and Linux kernel Bugzilla trackers, and issues from the Kubernetes GitHub repository, are cleaned, privacy-scrubbed, stripped of text that leaks their labels, and truncated to 2,000 characters. Each report then passes through the same three-step decision workflow, and the workflow's action is compared with a target action computed from the tracker's reference labels, which the model never sees. The comparison defines the failure label on which every signal is evaluated.

3.2 Workflow, repeats and action rule

All Jev calls go to the TypeSafe API with model `jev-latest`, which resolved to `jev-1.13.0` in every analysed call. Node 1 (`choice`) routes the report to a subsystem: network or storage on the Bugzilla trackers. Node 2 (`noul`) answers a binary question: customer impact on constructed data, regression on real bugs. Node 3 (`score`) rates severity on three levels. Each node reads the evidence plus the upstream decisions, appended by a fixed adapter. Each node is called three times, and the three answers are reduced to one decision: for `choice` and `score` we take the modal decision with a tie tolerance of 0.02, and a node with no two-of-three majority is *repeat-unstable*; for `noul` we threshold the mean probability at 0.5, and its confidence-like value is $2\,|\bar{p} - 0.5|$. The action rule F maps an *unclear* c_2 to *request more evidence*, $c_3 = 2$ to *escalate*, $c_3 = 1$ with $c_2 = 1$ to *defer*, and everything else to *accept* (Table 8).

3.3 Target action and failure signals

The target is $a^* = F(c_2^*, c_3^*)$, the same rule applied to the reference labels. F is a construct we defined, not an independently validated outcome, and every failure result is conditional on it. A workflow fails on a report when its action differs from the target, $y = \mathbf{1}[a \neq a^*]$. We compare three kinds of failure signal. The first is the model's own *final-step confidence* (low confidence ranks first), alone or composed across nodes. The second is the *action-rule score* $r(a) = (e(a) + \bar{r})/(n(a) + 1)$, the historical error rate of the action the workflow chose, where $e(a)$ and $n(a)$ count the errors and uses of action a on development data and $\bar{r}$ is the overall development error rate. The third is a frozen logistic estimator on the decisions (E_{dec}) or on the decisions plus the uncertainty signals (E_{all}), fitted on development data only.

3.4 Escalation strategies

The exploratory analyses compare six escalation strategies. S1 applies F to the node decisions, and S4a adds escalation on repeat instability. S2 escalates the $\lceil qn \rceil$ instances with the lowest product of per-node confidences and accepts the rest, where q is the share of non-accept targets, and S5 does the same with per-primitive rank-normalized uncertainty. Strategies 2h and 5h keep S1 and add confidence-ranked escalation of a fixed 10% budget, and R escalates the same budget at random.

3.5 Evaluation and statistics

Outcomes are $P(Y_{\text{decision}})$, the probability that the action equals a^*, and the cost under a frozen matrix (Table 9), in which a wrong *accept* costs 6–10, any other mismatch 1, and each escalation an extra 1 for human time. Agreement rates carry Wilson 95% intervals. AUROCs use the Mann–Whitney estimator, with 95% percentile bootstrap intervals over bugs ($B = 2{,}000$, fixed seed). Bugs are distinct reports with no shared unit, so we resample bugs. Paired strategy comparisons use the exact McNemar test on discordant pairs, and the difference in $P(Y_{\text{decision}})$ gets a bootstrap interval. Within every resample, the set-relative rankings (S2, 2h) are recomputed. We compare the two trackers through the difference of their paired differences, with independent bootstraps per tracker, and do not compare their p-values. For budgeted overrides, the reference is the *exact* expected cost of k random overrides; the randomization p-value (10,000 random k-subsets) is conditional on the evaluated bugs, and a bootstrap interval over bugs gives the sampling uncertainty. An interval that covers the null value is read as *no evidence of an effect*, not as evidence of no effect. Confirmatory tests are one-sided paired bootstrap tests with Holm correction within each pre-registered family. The scripts are `paper_statistics.py` and `mozilla_exploratory_analysis.py`.

Table 1: Methodological elements of this study and what each makes it possible to conclude.

Element	What we do	What it makes possible
Unit of analysis	A dependent chain of three structurally different typed decisions (categorical, binary, ordinal) mapped to an action by an explicit rule F	Failure is defined on the action taken, so every step-level signal can be compared as a predictor of workflow failure
Failure signals	Final-step and composed confidence, the action-rule score $r(a)$ and frozen estimators, compared by paired tests	Shows whether confidence adds anything beyond what the workflow decided
Prediction vs. value	A fixed 10% review budget scored with a frozen cost matrix against random review, confidence-ranked review and an oracle bound	Separates "ranks failures" from "finds the reviews that pay"
Shift diagnosis	Development and deployment label mixes differ by design; re-weighting with known and label-free (BBSE) mix estimates	Exposes the dependence of decision-conditioned error rates on the label prior
Confirmatory control	Pre-registration, hash-frozen estimators and protocols, dated amendments, independent recomputation	Separates confirmatory from exploratory claims (Table 10)
Replication	Five models behind one interface, and an independent, chronologically split Kubernetes workflow	Tests whether the findings belong to Jev or to the workflow design

3.6 Novelty of the methodology

Table 1 summarizes the methodological elements of this study and what each one lets us conclude. Together, the pre-registration, the cross-model and cross-workflow replications, and the split between ranking and value are what make us fairly confident that the negative result on review value is a real finding and not just a study that happened to come out flat.

4 Characterization: incompatible uncertainty semantics

Table 2 and Fig. 2 show three incompatible profiles. `choice`'s confidence is almost always near 1.0. It still discriminates, because the rare low-confidence answers are where its errors are. The high-confidence error subgroup found on deliberately ambiguous constructed items does not recur on real text (2 of 6 kernel errors are at ≥ 0.90). `noul`'s confidence-like value carries no usable information about correctness on either tracker. Its Mozilla interval covers 0.5, and its kernel interval lies at or just below 0.5 (weakly anti-informative). `score`'s confidence does not vary with the true level, so it gives no warning of the systematic bias in Section 6. Any composition rule that multiplies or averages these numbers inherits this mix.

5 Fixed thresholds lose their selectivity on real text

The rules in Fig. 3 fail in two different ways. On real text the trigger fired on 85% (Mozilla) and 96% (kernel) of bugs, and the product cut escalated 88% and 78% against targets of 41% and 42%

Table 2: Per-primitive behavior of the confidence signal. AUROC is for predicting the node's own agreement with the reference label, with 95% bootstrap intervals.

	Constructed items	Mozilla (n=202, triage)	Kernel (n=240, reporter)
`choice`	High-confidence error subgroup: mean confidence 0.915, accuracy 0.333 (n=6)	85% at ≥ 0.90, agreeing 0.96; AUROC 0.90 [0.83, 0.96]; ECE 0.05	96% at ≥ 0.90, agreeing 0.99; AUROC 0.98 [0.95, 1.00]; ECE 0.02
`noul`	24/24 correct, confidence below accuracy in every bin	Agreement flat across bands (0.71–0.84); AUROC 0.53 [0.42, 0.65]; ECE 0.36	Non-monotonic (0.58–0.93); AUROC 0.41 [0.32, 0.50]; ECE 0.36
`score`	Tracks the most severe sentence (6/6); confidence varies by 0.173 with sentence order alone (1 of 3 items)	Confidence ≈0.6 at every true level; AUROC 0.58 [0.50, 0.66]	Confidence 0.65 / 0.62 / 0.61 by true level; AUROC 0.57 [0.50, 0.64]

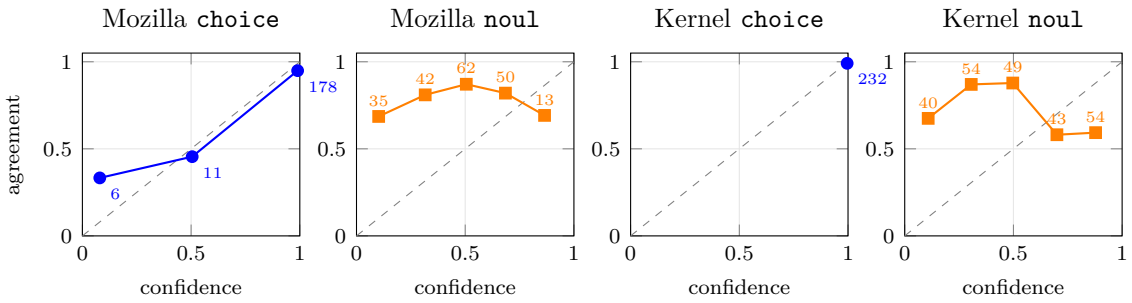


Figure 2: Reliability diagrams (five equal-width confidence bins, bins with $n \geq 5$; the label is n; dashed line: perfect calibration). `choice` puts nearly all bugs in its top bin, and that bin is accurate. `noul`'s confidence-like value $2|\bar{p} - 0.5|$ sits far *below* its agreement (underconfidence), and agreement does not rise with it.

(constructed pilot: 27/30 fired; 73% escalated against 67%). The `choice` trigger is non-selective *everywhere*, because `choice` confidence is saturated; this is a design problem that only shows up once you measure it. The product cut is *distribution-sensitive*: it was near its target on constructed data and far off on real data, because real text pulls `noul` toward 0.5, which drags the product down. A threshold just inherits the confidence scale of whatever data it was tuned on, and our constructed data was too clean to show the gap. The two Revision-10 strategies built on these thresholds scored 0.37–0.40 (Mozilla) and 0.34–0.37 (kernel). We replaced them with rank-based rules at a matched escalation rate before any evaluation.

6 Real-data validation on two trackers

Data. *Mozilla:* 202 fixed Firefox networking and storage bugs. Severity S1–S4 is assigned by triage and mapped to levels 2/2/1/0, and the regression flag is a keyword. Bugs whose severity was revised were excluded. *Linux kernel:* 240 fixed networking and storage bugs. Severity (blocking/high/normal/low → 2/2/1/0) is set by the reporter, and the regression flag is a dedicated field. Severity is rarely changed after filing: 15 of the 240 sampled bugs, all moving away from the default "normal". Both samples are balanced at 40 per (subsystem, level) cell where available. Texts are cleaned, privacy-scrubbed and truncated to 2,000 characters. Agreement is with the reference

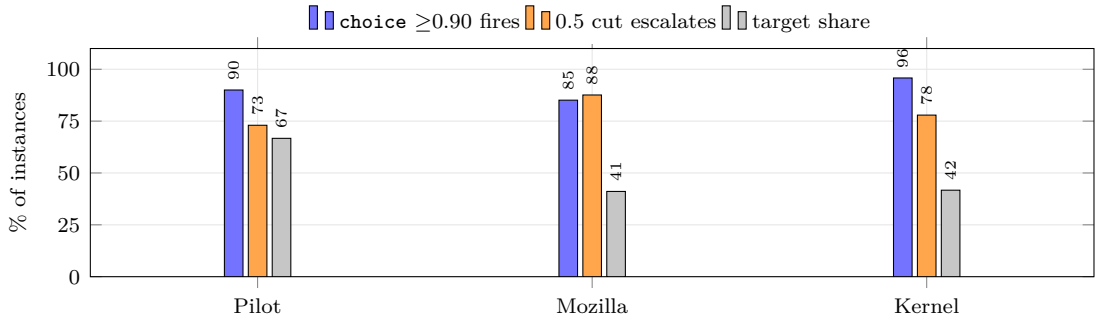


Figure 3: Selectivity of the two fixed-threshold rules, set on constructed data and measured on each rule's own input signals. The trigger fires on 85–96% of instances everywhere. The product cut is near its target on the constructed pilot and far above it on real text.

Table 3: Node agreement with reference labels (Wilson intervals). Each baseline is the most frequent label.

	Mozilla		Kernel	
Node	Agreement [95% CI]	Baseline	Agreement [95% CI]	Baseline
`choice`	0.89 [0.83, 0.92]	0.59	0.975 [0.95, 0.99]	0.50
`noul`	0.80 [0.74, 0.85]	0.74	0.725 [0.67, 0.78]	0.81
`score`	0.53 [0.46, 0.60]	0.40	0.44 [0.38, 0.50]	0.33

label, not ground truth.

Routing and regression. `choice` agrees with the component label well above baseline on both trackers (Table 3). It still agrees 0.84 on Mozilla texts with no subsystem keyword. `noul`'s errors change direction between trackers. On Mozilla it mostly misses regressions (23 misses vs. 17 false alarms). On the kernel it over-flags them (63 false alarms vs. 3 misses), which leaves it below the always-"no" baseline. The kernel false-alarm rate is the same on bugs explicitly marked "not a regression" (33/100) as on bugs with the field unset (30/94). So unset labels do not explain it, and on bugs with a set field `noul` beats its baseline (110/146 vs. 100/146).

Severity compression. Figure 4 shows the row-normalized confusion of `score`. On both trackers, most minor bugs are rated higher than their label: 79% [0.67, 0.88] on Mozilla and 75% [0.65, 0.83] on the kernel. A large share of serious bugs are rated lower: 43% and 56%. The middle level absorbs both. With Mozilla's own severity definitions in place of ours, serious bugs improve but normal bugs are pushed up, and agreement moves only from 0.53 to 0.56. So the exact criteria do change the numbers, but the compression is still there. The two trackers differ in project, bug population and labeling process. They share Jev, the prompts, our sampling design and, plausibly, training-data exposure, so the replication speaks against an artifact of one *labeling* process, not against every shared factor. The pattern matches central-tendency bias reported for LLM raters in other domains [44].

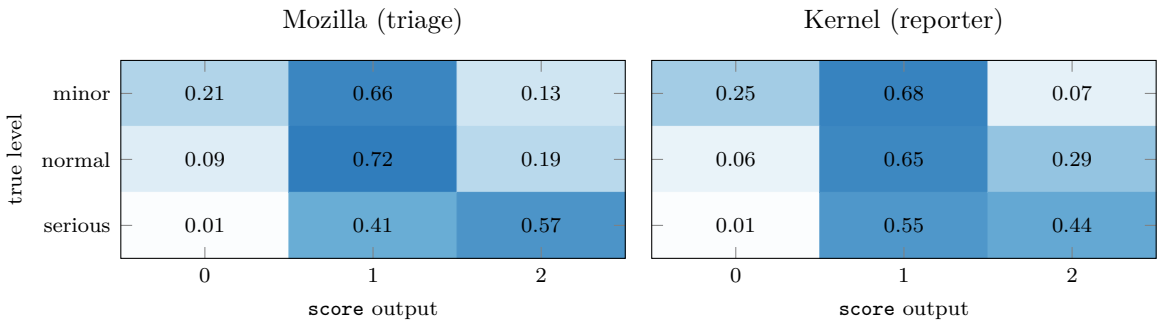


Figure 4: Row-normalized confusion of `score` against reference severity (repeat-unstable outputs excluded). On both trackers the middle column absorbs mass from both extremes: minor bugs are rated up, serious bugs down.

7 Composition (exploratory)

All analyses in this section were specified after the Mozilla results had been seen. They are hypothesis-generating. The pre-registered tests that followed are reported in Sections 8, 9 and 11.

The naive comparison is confounded. On Mozilla, decision composition (S1) beats confidence composition (S2) by 0.17 [0.08, 0.27] in $P(Y_{\text{decision}})$, with 61 vs. 27 discordant pairs (exact McNemar $p = 0.0004$). But S2 never reads the node decisions, while a^* escalates on the severity *decision*. To be right on a serious bug, S2 needs low confidence to *mean* high severity, so the comparison mostly measures whether confidence can stand in for severity. On the kernel the difference is smaller and inconclusive: 0.06 [−0.03, 0.15], 66 vs. 51 discordant pairs, $p = 0.20$. The difference between trackers is 0.11 [−0.01, 0.24]. Its interval covers zero, so we have *no evidence that the effect differs* between trackers, only a weaker estimate on one of them.

The fair question, and two different targets. Does confidence add anything *beyond* the decisions? Starting from the decision-based action F, we also escalate the k least-confident bugs that F did not escalate, for every k, and compare with the exact expectation under random selection (Fig. 6). We report two AUROCs with different prediction targets, which should not be conflated. Target (A) is *missed serious bugs*: among bugs that F did not escalate, does low composed confidence rank those whose target is *escalate* first? This is what an override budget can fix, and the AUROC is 0.49 [0.37, 0.61] on Mozilla and 0.50 [0.40, 0.60] on the kernel. Target (B) is *any decision error*: across all bugs, does low composed confidence rank those where F's action is wrong first? The AUROC is 0.54 [0.45, 0.63] on Mozilla and 0.47 [0.40, 0.54] on the kernel. Every interval covers 0.5 (Fig. 5). The data show no evidence that composed confidence ranks either target better than chance, but they cannot rule out a modest signal (AUROC up to about 0.6).

Budgeted overrides, scored by cost. At a fixed budget of $k = \lceil 0.1n \rceil$ overrides ($k = 21$ on Mozilla, 24 on the kernel), let T be the total cost of the confidence-ranked overrides and μ the exact expected total under k overrides drawn uniformly from the same candidates. Two analyses answer two different questions. A randomization test, conditional on the evaluated bugs, asks whether the ranking selected worse or better overrides than random selection *on these bugs*; its statistic is $|T - \mu|$, and its two-sided null distribution comes from 10,000 random k-subsets of the candidates. A bootstrap over bugs asks how stable the per-bug difference $(T - \mu)/n$ is under resampling of the

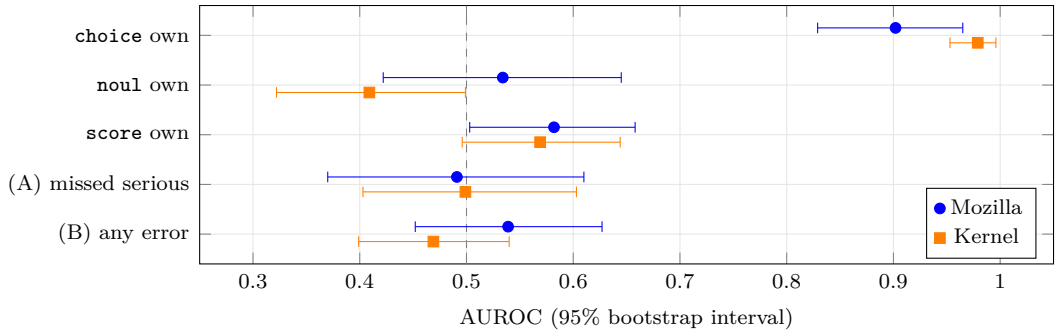


Figure 5: Every AUROC in the paper with its interval; dashed line: chance. Only `choice`'s confidence discriminates its own errors. Composed confidence is at chance for both escalation targets, (A) and (B).

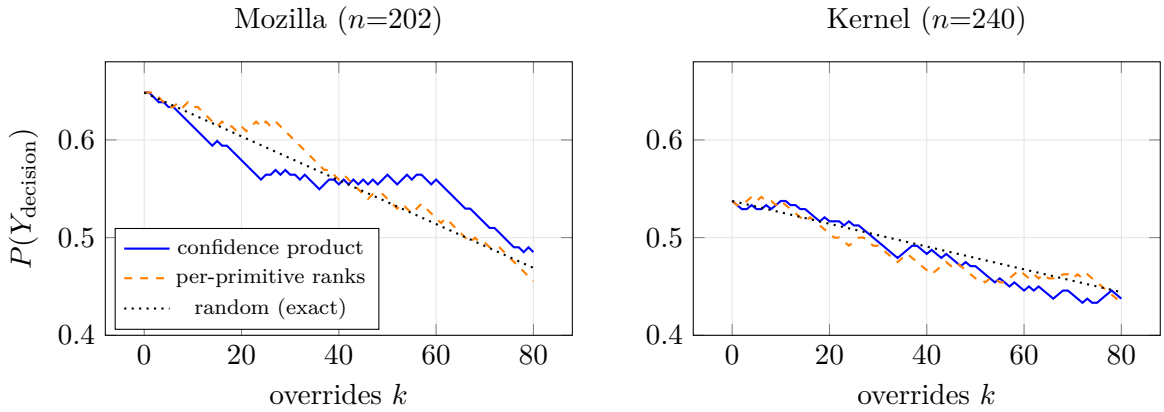


Figure 6: Adding confidence-ranked escalations on top of decision composition. At $k = 0$ the curve is S1. Neither ranking consistently improves on random selection of the same number of cases: deviations are a few points at most and change sign between rankings and trackers. For example, per-primitive ranks run slightly above random on Mozilla for $k \approx 15$–35, and the confidence product runs below.

bugs, with rankings and μ recomputed in each resample. On Mozilla, $T = 390$ against $\mu = 357.2$. The observed value sits at the edge of the null distribution (2.5–97.5% range 323–388), so $p = 0.05$. The per-bug difference is +0.16 [−0.01, 0.24], and 97% of resamples are positive. That is borderline evidence that the product ranking chose *worse* overrides than random on Mozilla. On the kernel, $T = 338$ against $\mu = 358.6$ ($p = 0.18$), a per-bug difference of −0.09 [−0.23, 0.06]: inconclusive, and in the opposite direction. These analyses are exploratory and uncorrected for multiplicity. We read them as no consistent benefit from confidence-ranked overrides, with one borderline harmful estimate that did not replicate, and not as an established harm.

A structural reason. In this workflow the only confidence that actually discriminates (`choice`, AUROC 0.90–0.98) sits on the one node that F never uses. So the useful signal does not reach the action at all, and the two signals that do reach it carry almost no information.

8 A pre-registered held-out test

Design. The exploratory analyses above suggested that a model's *decisions* carry the information its confidence lacks. We tested this prospectively. We trained one decision-feature estimator (E_{dec}: L_2-regularized logistic regression on the `score` level, the `noul` decision and the workflow action, with $\lambda = 1$) on all 442 previously collected bugs and froze it by hash before any new data existed. We then collected 500 new bugs, 250 per tracker, sampled at each tracker's *natural* mix with no overlap with the training bugs, which required 4,500 Jev calls. Two hypotheses were pre-registered: E_{dec} ranks wrong workflow actions better than low final-step confidence (HR1), and E_{dec} ranks above chance on each tracker (HR2), both by one-sided bootstrap tests with Holm correction.

Result. Both hypotheses are supported. For HR1, E_{dec} reaches AUROC 0.763 against 0.549 for final-step confidence, a difference of +0.214 (one-sided 95% lower bound +0.158). For HR2, the AUROC is 0.811 on Mozilla (lower bound 0.762) and 0.697 on the kernel (lower bound 0.648). Per tracker, the difference is +0.279 [+0.191, +0.374] on Mozilla and +0.124 [+0.023, +0.225] on the kernel, weaker but excluding zero.

The result survives three integrity checks. Tracker links (duplicates, see-also, blocks, depends-on, regressions) connect 9 held-out bugs to development bugs; without them E_{dec} reaches 0.770 against 0.551. On the 70 Mozilla bugs filed in the 12 months before sampling, E_{dec} reaches 0.847 against 0.519. Refitting the estimator after removing every development bug linked to a held-out bug gives 0.772 against 0.551 (difference +0.221 [+0.154, +0.288]). A separate script that shares no code with the analysis rebuilds the reduction, action rule, features, AUROC and bootstrap from the raw responses and reproduces every point estimate exactly. The estimator's predicted failure rate matches the observed rate overall, but it outputs only 7–8 distinct risk levels (ECE 0.18–0.20).

9 Cross-model replication

Models. To test whether the result is specific to Jev, we ran the same 942 bugs, questions and three-step adapter through four further models. OpenJev 27B [32] is an independently trained Jev-style decision model (Hugging Face `openjev/openjev`, commit `b4a89f5`; several unrelated projects share the name), served on one H100 exactly as its authors specify, with their calibrated readout; its 2,826 requests had 0 failures. (An unofficial architectural reconstruction of Jev also exists [20], but it has no trained weights, so it could not be evaluated.) The other three are general instruct models, Qwen2.5-7B [35], OLMo-2-7B [39] and Phi-3.5-mini [1], each run behind a Jev-compatible readout: next-token probabilities over the allowed labels (as in [19]), with TypeSafe's documented confidence formulas. Each model's estimator was trained on that model's own outputs for the 442 training bugs and tested on its 500 held-out bugs. The pre-registered tests mirror HR1 and HR2, with Holm correction across all 8 tests.

Why these models. Jev is itself a language-model-based decision service used zero-shot: it receives text and a typed question, with no training on the task. The comparison that isolates Jev is therefore other models used the same way, through the same questions, adapter and confidence formulas. We chose open-weight models at the small end of zero-shot classification, 3.8B to 7B parameters, the scale chosen when routine classification must be cheap, from three independent families with different training data, plus OpenJev, the only openly available model trained for Jev's interface (27B). Open weights make every run reproducible. The question is not which model classifies best, but whether the failure-prediction findings belong to Jev or to any model that makes

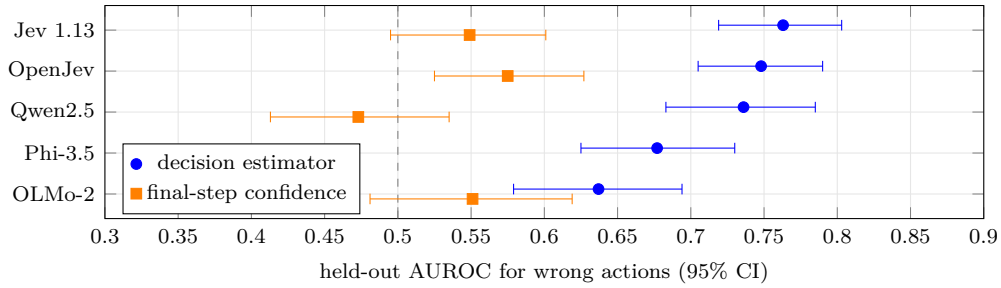


Figure 7: Across five models, the decision-feature estimator ranks wrong workflow actions better than the model's own final-step confidence. The intervals overlap only for OLMo-2.

Table 4: Per-model results on the same 500 held-out bugs. AUROC columns rank wrong workflow actions (95% bootstrap CI): the decision-feature estimator fitted on each model's own training outputs, the model's final-step confidence, the action-conditional error-rate rule, and Jev's frozen estimator applied unchanged. Cost columns are from the pre-registered intervention test (Section 11); the oracle is an upper bound that must spend all 50 reviews, and "wrong accepts" is their share of no-review cost. Absolute failure rates are not a clean quality comparison, because the Jev-style and general models use different interfaces.

			Held-out AUROC for wrong actions				Cost at 10% review budget			
Model	Type	Fail rate	Decision est.	Final-step conf.	Action rule	Jev est.	No review	Best review arm	Oracle	Wrong accepts
Jev	TypeSafe, hosted	0.43	0.76 [0.72, 0.80]	0.55 [0.49, 0.60]	0.76	—	658	681 (conf. product)	370	52%
OpenJev	Jev-style, 27B	0.47	0.75 [0.70, 0.79]	0.57 [0.53, 0.63]	0.75	0.76	660	674 (confidence)	441	40%
Qwen2.5	general, 7B	0.74	0.74 [0.68, 0.79]	0.47 [0.41, 0.54]	0.70	0.74	903	910 (estimator)	856	14%
Phi-3.5	general, 3.8B	0.72	0.68 [0.62, 0.73]	0.40 [0.34, 0.46]	0.69	0.70	831	890 (confidence)	890	4%
OLMo-2	general, 7B	0.80	0.64 [0.58, 0.69]	0.55 [0.48, 0.62]	0.58	0.64	893	916 (all arms tie)	916	1%

typed decisions through this interface; weaker models are useful for that, since they produce more failures to predict. A supervised classifier trained on each workflow's labels, such as a linear model on bag-of-words features, is the simpler alternative wherever such labels exist; it answers a different question and is not evaluated here.

Deviations, disclosed. We recorded three pre-registration amendments. OpenJev was added before any of its outputs existed; OLMo-2 needed a newer `transformers` release; and a readout bug, in which a SentencePiece word-start token was shared by all digit labels, was found after interim results had been seen, so all three general models were re-run under one corrected readout. The conclusions did not change.

Result. All 8 pre-registered tests are supported (Fig. 7, Table 4). The decision estimators reach AUROC 0.64–0.76 and final-step confidence 0.40–0.58. The weakest model is OLMo-2 (0.637 vs. 0.551; one-sided $p = 0.037$, passing at the last Holm step). Jev's frozen estimator, never refitted, reaches 0.76 [0.72, 0.80] on OpenJev's outputs, as well as OpenJev's own estimator does.

The action's historical error rate carries the signal. A class-frequency rule, "the historical error rate of the action the workflow chose" (estimated on training bugs only), matches the estimator for Jev (0.76) and OpenJev (0.75), and comes close for the others (0.58–0.70). Single uncertainty signals each sit at chance for workflow failure: routing confidence at 0.48–0.52, regression confidence at 0.45–0.56 and severity entropy at 0.40–0.55. The predictive information is therefore mostly *which action was chosen* and how often that action has been wrong before. Part of this is structural:

Table 5: Exploratory: held-out AUROC of the decision-only estimator on two targets that do not pass through the action rule F, fitted on the quota-sampled development bugs ("dev fit") or on the same bugs re-weighted to the natural label mix, against final-step confidence. Bold: re-weighted estimator above confidence.

	Severity wrong			Any node wrong		
Model	dev fit	re-weighted	conf.	dev fit	re-weighted	conf.
Jev	0.28	**0.79**	0.61	0.56	**0.76**	0.56
OpenJev	0.28	**0.79**	0.63	0.62	**0.81**	0.56
Qwen2.5	0.62	**0.74**	0.46	0.76	**0.73**	0.40
Phi-3.5	0.31	**0.67**	0.41	0.67	**0.77**	0.29
OLMo-2	0.50	**0.62**	0.57	0.42	0.42	0.49

the action and the target are both outputs of the same rule F, so an action's error rate is a property of the workflow as much as of the model. Paired bootstrap contrasts make this precise. For Jev and OpenJev the decision estimator adds nothing detectable beyond the action rule (+0.000 [−0.012, +0.011] and +0.002 [−0.007, +0.011]); for Qwen2.5 and OLMo-2 it adds a little (+0.035 and +0.057, intervals excluding 0). Adding the uncertainty features to the decision features raises held-out AUROC by +0.006 (Jev), +0.004 (OpenJev) and +0.012 (Qwen2.5), none distinguishable from 0, but by +0.054 (Phi-3.5) and +0.078 (OLMo-2), both with intervals excluding 0. For the Jev-style models confidence is redundant with the decisions; for the weaker general models raw token probabilities carry some extra information.

Beyond the constructed target (exploratory). Because the target a^* is a construct, we also asked whether decisions predict failures on two targets that bypass F: the severity answer being wrong, and any node being wrong (Table 5). Fitted on the development bugs, several decision estimators rank held-out severity errors far *below* chance (0.28 for Jev and OpenJev). The cause is label-prior shift: a decision's error rate depends on the label mix. In the quota-sampled development set Jev's level-1 answers are 60% wrong; in the natural-mix held-out set, where most bugs are level 1, they are 20% wrong. Re-weighting the development bugs to the natural label mix repairs this: the decision estimator then beats final-step confidence on both targets for four of five models (Jev 0.79 vs. 0.61 on severity), the exception being OLMo-2 on any node. These oracle weights use the held-out sample's label counts. A deployment-feasible version uses no test labels: black-box shift estimation [27] infers the deployment severity mix from the development confusion matrix of each model's `score` answers and the distribution of its answers on the new bugs. It recovers almost all of the oracle gain for Jev (0.74 on severity, 0.78 on any node), OpenJev (0.78, 0.80) and Qwen2.5 (0.74 on severity), and none for Phi-3.5 and OLMo-2, whose severity answers are nearly all "blocking", so their confusion matrices are near-singular and their answers carry no information about the label mix. Decision-conditioned error rates are therefore informative beyond the constructed target, but only when estimated under the deployment label distribution, which can be estimated without labels only for models whose answers discriminate the label. Two caveats bound this. Under known synthetic shifts, the label-free estimate of the mix is biased toward the development mix (for Jev it underestimates a 50% share of serious bugs by 0.21), so BBSE does not recover the deployment mix. The ranking improves anyway because the estimate collapses onto the middle severity level

(for the kernel it is $[0, 1, 0]$ against a true $[0.02, 0.73, 0.25]$): the re-weighted fit is then dominated by middle-level bugs, as the natural-mix set is, so the decision-to-error relationship takes the right direction even though the other levels are effectively dropped. Where the estimate points the wrong way, the repair fails: for Phi-3.5 it puts no weight on the middle level, and ranking does not improve. In short, the ranking improved for Jev, OpenJev and Qwen2.5, but the estimate of the deployment mix itself did not, and whether the ranking gain survives deployment mixes in which the middle level is not dominant is still open, since it may depend on that coincidence. In other words, the correction made the ranking better even though it never actually estimated the label mix it was supposed to estimate. And the constructed-target result itself is not fragile to how labels are mapped: with Mozilla's S2 counted as level 1, or kernel bugs with default severity or unset regression excluded, decisions still beat confidence for every model, except OLMo-2 when default-severity kernel bugs are removed.

Failure modes differ by model family. The two Jev-style models, Jev and OpenJev, both pull severity to the middle level. The general instruct models push severity to "blocking" for 67–100% of bugs, whatever the true level, and are wrong far more often (72–80% vs. 43–47%). Because the two groups use different interfaces, the absolute failure rates are not a clean quality comparison, but the within-model contrast between the estimator and confidence is.

10 An independent workflow: Kubernetes issue triage

Design. To test whether the predictive finding is a property of our bug-triage setup, we pre-registered a replication in an independent workflow before any of its data existed: issue triage in the Kubernetes GitHub repository. It differs in platform, labelling process (triager-applied priority and SIG labels, reporter-chosen bug/feature forms), three-way routing (network, storage, node), the binary question (bug or not) and the ordinal question (priority). The split is chronological: development on 500 issues from 2015–2019, test on the 582 latest eligible issues from 2021–2026, with linked issues removed across the split. Leakage of labels into the text (bot commands, template headings, team mentions) was removed and audited, and `jev-1.13.0` was pinned. All of the protocol's seven amendments, including a one-time widening of the test window after the first feasibility check failed, were written before any model output existed. Development and test differ at once in period, issue-form use and label prevalence, so this is a combined test of temporal and distribution shift, not of label shift alone.

Result. The primary hypothesis is supported: the chosen action's historical error rate ranks test-set failures better than final-step confidence, 0.776 against 0.628, a paired difference of +0.148 [+0.091, +0.207], above chance as well. As in the original study, the signal is almost entirely the action: the decision estimator adds a little (+0.004 to +0.017) and confidence adds nothing. The contrast holds for the any-node target, within SIG Node, and on issues triaged within 30 days.

Four further models. A second pre-registration, written before any of their outputs existed, ran the same workflow and analysis on the four comparison models (1,082 issues each, no failures), with Holm correction over all eight primary tests. Here the models split by competence. Only the two models trained for typed decisions beat chance on every step (routing, bug or not, and priority): Jev and OpenJev-27B, whose priority accuracies were 0.371 and 0.404 against a chance level of one third. For both, the action rule beat final-step confidence (OpenJev-27B: 0.796 vs. 0.569). The three general instruct models handled routing and the bug question but not priority (accuracy 5–27%), so

their workflows were wrong on most issues (wrong-action rates up to 95%); this competence check was added after the results were seen, applied to every model, and does not replace the primary result. Among these weak models the rule still beat confidence for Qwen2.5 (0.710 vs. 0.555) and OLMo-2 (0.850 vs. 0.255), but largely by flagging the one action each model chose almost always; it was not replicated for Phi-3.5 (0.519 vs. 0.674), which escalated 580 of the 582 test issues and so left the rule nothing to rank. By the pre-registered rule (at least three of four) the finding holds on Kubernetes.

The label-free correction did not transfer. The pre-registered label-shift test is inconclusive by its frozen rule, with a negative estimate: re-weighting a deliberately mismatched fit by BBSE lowered its AUROC from 0.699 to 0.348 (-0.350 [-0.428, -0.271]). The mechanism is the one identified above. BBSE was numerically stable (condition number 39) but again estimated the deployment mix as entirely middle level, $[0, 1, 0]$, whereas the true mix was $[0.69, 0.26, 0.05]$, dominated by the lowest level, so the weighting pushed the fit the wrong way. The original BBSE gains therefore depended on the middle level being dominant.

11 Does the predictive signal reduce cost?

Design. We pre-registered the test before computing any cost outcome. Each model gets a budget of 50 reviews (10%) on the held-out bugs; a review escalates the case to a human and is priced with the frozen cost matrix. The primary arm (A1) ranks candidates by the *action-level expected saving* of a review, estimated on training bugs; it is cost-aware rather than error-rate-based, because a wrong *defer* costs no more than an escalation. The comparators are confidence-ranked review (B) and random review (C).

Result. Two of ten pre-registered tests are supported: A1 beats random for Jev and OpenJev, by about 10–12 cost units. A1 is not shown to beat confidence for any model (for Jev its point estimate is 4 units lower, with an interval spanning zero). Under the frozen table, every review arm costs more than no review at all; for Jev, 658 with no review against 681–714 with review (Fig. 8).

Sensitivity to costs (exploratory). The pre-registered result holds under one frozen cost table. In a post-hoc analysis planned before computing it, we re-scored the same frozen rankings on a grid of cost ratios within a factor of 2 of that table (Table 6). The conclusion is cost-robust only for OLMo-2. For Jev, OpenJev and Qwen2.5, review pays when wrong accepts cost twice as much (for Jev, 44–69 units below no review) or when review costs half as much, because the break-even precision p^* then falls to about the 14–16% share of wrongly accepted cases. Some reversals are small (Jev -2 at $m = 1$, $h = 0.5$) and lie well inside the intervals of the original tests. The ordering is more stable than the sign: the action rule beats random for both Jev-style models in 12 of 16 cells. Imperfect reviewers shrink but do not reverse these savings: if a review resolves correctly only 80% of the time, Jev's saving at twice the wrong-accept cost falls from 44 to 26 units. The stopping decision below was made under the frozen table and, by its rule, is not reopened.

Where the cost is. A descriptive follow-up separates two regimes. For the Jev-style models, 40–52% of no-review cost comes from wrongly *accepted* serious or regression bugs, and an oracle that picked the right 50 cases would save 33–44%. But those cases are 15–16% of accepted cases. The action rule cannot separate them, because every accept looks alike to it, and confidence cannot either: among Jev's 245 accepted held-out cases, the decision estimator, final-step confidence and

Table 6: Exploratory cost sensitivity: best feasible arm minus no review (negative, in bold: review pays), with rankings frozen as pre-registered. m scales wrong-accept costs, h is the human cost per escalation, and p^* is the break-even precision for reviewing an accepted case whose target is *escalate*. †The pre-registered table. Point estimates only.

m	h	p^*	Jev	OpenJev	Qwen2.5	Phi-3.5	OLMo-2
0.5	0.5	25%	+29	+25	+21	+52	+20
0.5	1	33%	+54	+50	+46	+77	+28
0.5	2	50%	+104	+100	+96	+127	+45
1	0.5	14%	**-2**	**-11**	**-18**	+34	+14
1	1†	18%	+23	+14	+7	+59	+23
1	2	27%	+73	+64	+57	+109	+40
2	0.5	7%	**-69**	**-83**	**-97**	**-2**	+4
2	1	10%	**-44**	**-58**	**-72**	+23	+13
2	2	14%	+6	**-8**	**-22**	+73	+30

`score`'s probability of the top level rank the 38 wrong accepts at AUROC 0.46, 0.49 and 0.49, with 95% intervals that all include chance (for example 0.49 [0.37, 0.60] for confidence). With so few positives this shows that the tested signals did not establish useful discrimination for Jev, not that none is possible; for OpenJev, `score`'s top-level probability does carry a weak within-action signal (0.68 [0.56, 0.78]), and a paired comparison puts it above confidence by +0.14 (95% interval [+0.04, +0.22]). So the recoverable value exists for OpenJev, but for Jev the tested signals do not isolate it. For the general models, cost is dominated by over-escalation (*escalate* when the target is *accept*), which more review cannot reduce; even the oracle saves at most 5%. So being able to rank which actions are wrong does not get you the reviews that actually save money; the errors that happen most often are not the ones that cost the most.

A case-level score, and a stopping decision. We then asked whether a per-case score could find the expensive errors. It was developed on the 442 training bugs only, under a plan committed before any score was computed. A parameter-free plug-in score was selected by a pre-stated rule over a small logistic model and frozen. It ranks accepted cases by expected saving from review, $9p_2 + 6p_1q - 2(1 - p_2 - p_1q)$, using Jev's `score` mass on levels 1 and 2 (p_1, p_2) and its `noul` probability q. On the training mix it cut cost by 12% against no review (630 vs. 713) and captured 22% of the oracle saving. But the training bugs were quota-sampled. Reweighted to each tracker's natural label mix, a 10% budget is projected to cost *more* than no review, with power near zero. Before generating any fresh case, we pre-registered a feasibility check that varied only the review budget (1–10%). Viability required power ≥ 0.80 to show lower cost than no review, with at most 1,000 fresh bugs, and a negative median projected cost change; each simulated study first resampled the training bugs, so uncertainty in error prevalence and ranking quality was propagated. No budget was viable. The highest power was 0.20 (1% budget, 1,000 bugs), with a median change of -2 and an 80% range of $[-46, +20]$, and every budget of 2% or more had a positive median cost change and power below 0.15. By the binding stop rule, the confirmatory study was not run, no further designs were searched, and no fresh API calls were made. This was a feasibility check, so it does



Figure 8: Total cost at a 10% review budget under the pre-registered cost table. No feasible strategy beats no review. A perfect reviewer (oracle, an upper bound using held-out outcomes) would cut Jev's cost by 44% and OpenJev's by 33%. For the general models it saves little or nothing: they escalate most cases themselves, leaving few reviewable errors (Phi-3.5's oracle exceeds no review because 50 reviews must be spent on 63 candidates, only 4 of which gain).

not show that review can never help. The oracle makes clear that a good selector would save a lot; the point is that the signals we had were not sharp enough to pick out the cases that would pay for the reviews, at least under the costs we fixed in advance.

12 Lessons from the study's own audits

Table 7 lists the defects the study's own audits caught. Five of seven were found before the data that would have exposed them existed. The exceptions are the confound, found after a result, and the bootstrap seed, found by re-running every analysis from a clean checkout; it moved two lower bounds by at most 0.005 and changed no test outcome. We would have caught it earlier if, for every strategy and every target class, we had asked whether that strategy could even be right there and through what information. We suggest running that check, along with measuring each trigger's selectivity on pilot inputs, before freezing an escalation policy.

13 Threats to validity

Construct validity. The failure label is disagreement with a constructed target, $a^* = F(\cdot)$, and all strategy results are conditional on it. Its inputs are tracker labels, which are not truth: Mozilla's triage sample excludes re-triaged bugs, which biases agreement upward, kernel labels are reporters' claims, and the kernel's middle level is a default value. An exploratory pilot in which one AI assistant chose the right action for 50 held-out bugs, blind to labels and outputs, agreed with the constructed target at about chance level (Cohen's κ between -0.157 and $+0.015$ under two pre-stated readings of its categories). With a single rater and no inter-rater baseline, this cannot tell whether the target or the judge is at fault, but it makes the limitation concrete: every failure result in this paper is disagreement with an operational target built from tracker labels, and its correspondence to the action a competent evaluator would choose is unestablished. A pre-registered two-human validation is designed and not yet run. The review-value analysis uses one budget and one cost table, both frozen in advance; real operational costs are unknown, reviewers are assumed correct, and the oracle is an upper bound, not a strategy.

Internal validity. The 500 held-out bugs were fresh for the prediction tests only. The intervention test was pre-registered after their prediction results were known (no cost outcome had been

Table 7: Defects found by the study's own audits.

Defect	Detected by	When
A "primitive-aware" arm structurally identical to the baseline	inspecting the action rule	before evaluation
A trigger firing on ≈90% of inputs	selectivity on input signals only	before evaluation
A threshold sensitive to the data distribution	smoke test vs. pilot	before the full run
Four silent runner defects (e.g. the literal "None" shown to the model; partial data analyzed)	adversarial probing	before the live run
The confidence-vs-decisions confound	post-hoc review	after the Mozilla result
Template content whose labels were recoverable from punctuation alone (100%) and whose sentences leaked cross-label cues	independent review (separate AI agent)	before the controlled study
A bootstrap seed derived from Python's salted string hash, making two reported lower bounds vary between runs	clean-checkout re-run of all analyses	after publication of the bounds

computed), and the cost-sensitivity analysis is post hoc on the same bugs, so neither is an independent confirmation. Decision-conditioned error estimators fitted on the quota-sampled development bugs can invert on a natural-mix set (Table 5); the label-free correction (BBSE) corrects only the severity dimension, assumes pure label shift and fails by construction for models whose answers do not discriminate the label, and these analyses are exploratory and on the same held-out bugs. The constructed-data findings rest on small samples (the `choice` error subgroup is $n = 6$, and `noul`'s perfect accuracy on constructed data is 24 items). Old public bugs may appear in model training data, and the recent subsets used to check this are small (Mozilla 54, kernel 35). The protocol went through twelve revisions and the cross-model protocol three amendments, including a readout bug fixed after interim results were seen; on 3% of OLMo-2's requests most of the probability fell outside the allowed labels, and excluding them does not change its result. All revisions are dated and disclosed, and each was made before the data it affects existed, though twelve is still a lot of revisions.

External validity. The study covers five models in one domain, software issue triage, across two workflows (Bugzilla bug triage and Kubernetes issue triage), with Jev at version 1.13.0. The general instruct models use our prompt and readout, not a trained decision interface, and on Kubernetes they were below chance on the priority step. The Kubernetes study is one further workflow: its development and test sets differ in period, format and label mix at once, its leakage read was performed by an AI assistant at the authors' direction rather than by a human author as

pre-registered, and its protocol has seven dated amendments, all written before any model output existed. Later model versions and other domains may behave differently.

14 Conclusion

We started with two questions: can the uncertainty signals in a chained, typed decision workflow tell you where it fails, and can they tell you which failures are worth a human's time. On the first, what the workflow decided predicted its failures better than how confident it was, on 500 new bugs and across five models, with failure measured against the target we built, and most of that signal was just the historical error rate of the chosen action. It held up on a separate Kubernetes workflow across a five-year gap, for Jev and three of the four other models.

The catch is that these error rates depend on the label mix they were estimated under, and they can flip when that mix changes. Re-weighting to a known deployment mix fixes them, but the label-free way of estimating that mix only worked when the middle severity level dominated, and it made things worse on Kubernetes, where it did not.

On the second question the answer was no. Under the costs we fixed in advance, none of the review policies we built from these signals beat doing no review at all. The expensive mistakes, wrongly accepted serious bugs, look like the cheap ones in both their action and their confidence, so neither signal pulls them out, and whether review pays comes down to how much a wrong accept costs relative to a review. Predicting failures, estimating uncertainty, and deciding which reviews are worth it turned out to be three different problems, and we were only able to make real progress on the first.

15 Future scope

There are a few natural next steps. The clearest is to check the target itself: a pre-registered study where two human evaluators independently pick the right action for a sample of held-out bugs would show how close our constructed target is to expert judgment, and whether the gaps come from the tracker labels, the action rule, or real ambiguity. It would also help to have a signal that separates expensive errors *within* an action, since the oracle shows such a signal would make review worthwhile; OpenJev's weak within-action severity signal is a hint about where to look. The whole setup should also be tried on workflows outside software triage, on newer model versions, and against a supervised classifier as a baseline, to see how far the split between prediction and review value carries over, and review value should be measured against real costs with reviewers who can themselves be wrong. We have also pre-registered, but not yet run, a study on deterministic template content (3,103 units with value-neutral sentence frames and 13 automated audits) that tests the composition question under tighter control, comparing confidence-ranked against random overrides at a 10% budget scored by cost, and per-primitive against raw-product ranking. Given the results above it may well come back null, and nothing in this paper depends on it.

Acknowledgments

We used an AI assistant (Claude, Anthropic) for analysis code, for help with writing, and for the specific tasks noted where they occur (the Kubernetes leakage read, the target-validity pilot and an independent review of the template content). We directed the study, approved every pre-registration and every model run, and are responsible for the content.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Jean V. Alves, Diogo Leitão, Sérgio Jesus, Marco O. P. Sampaio, Javier Liébana, Pedro Saleiro, Mário A. T. Figueiredo, and Pedro Bizarro. Cost-sensitive learning to defer to multiple experts with workload constraints. *Transactions on Machine Learning Research*, 2024.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[4] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Varun Babbar, Umang Bhatt, and Adrian Weller. On the utility of prediction sets in human-AI teams. In *Proceedings of IJCAI*, 2022.

[6] Gagan Bansal, Besmira Nushi, Ece Kamar, Eric Horvitz, and Daniel S. Weld. Is the most accurate AI the best teammate? optimizing AI for teamwork. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11405–11414, 2021.

[7] Richard E. Barlow and Frank Proschan. *Mathematical Theory of Reliability*. Wiley, 1965.

[8] Dylan Bouchard. Is escalation worth it? On the depth of LLM cascades. *arXiv preprint arXiv:2605.06350*, 2026. v2; v1 was titled "Is Escalation Worth It? A Decision-Theoretic Characterization of LLM Cascades".

[9] Aritra Chowdhury, Malik Magdon-Ismail, and Bülent Yener. Quantifying contribution and propagation of error from computational steps, algorithms and hyperparameter choices in image classification pipelines. *arXiv preprint arXiv:1903.00405*, 2019.

[10] Vittorio Cortellessa and Vincenzo Grassi. A modeling approach to analyze the impact of error propagation on reliability of component-based systems. In *Component-Based Software Engineering (CBSE 2007)*, volume 4608 of *LNCS*, pages 140–156. Springer, 2007.

[11] Steven Denney and Matthew DiGiuseppe. JEV versus LLMs: Accuracy, cost and calibration on seven political science replications. *arXiv preprint arXiv:2610.06625*, 2026.

[12] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. Evaluating and benchmarking the system one model Jev. *arXiv preprint arXiv:2609.37647*, 2026.

[13] Yifan Dou, Shikan Lian, and Shibo Li. Conformal cascade: Distribution-free accuracy guarantees for multi-tier LLM inference. *arXiv preprint arXiv:2607.25018*, 2026.

[14] Jinhao Duan, James Diffenderfer, Sandeep Madireddy, Tianlong Chen, Bhavya Kailkhura, and Kaidi Xu. UProp: Investigating the uncertainty propagation of LLMs in multi-step agentic decision-making. *arXiv preprint arXiv:2506.17419*, 2025.

[15] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

[16] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4878–4887, 2017.

[17] Yonatan Geifman and Ran El-Yaniv. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2151–2159, 2019.

[18] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. A survey of confidence estimation and calibration in large language models. In *Proceedings of NAACL (Volume 1: Long Papers)*, pages 6577–6595, 2024.

[19] Jonathan Geuter, Andrew Cheng, and David Alvarez-Melis. Releasing the inner Jev: A simple recipe for open decision models. *arXiv preprint arXiv:2610.17205*, 2026.

[20] Kye Gomez. open-jev: an unofficial reconstruction of Jev's architecture. `https://github.com/kyegomez/open-jev`, 2026. Untrained architecture with random weights. Accessed 2026-10-17.

[21] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[22] Jaehun Jung, Faeze Brahman, and Yejin Choi. Trust or escalate: LLM judges with provable guarantees for human agreement. In *International Conference on Learning Representations (ICLR)*, 2025.

[23] Josef Kittler, Mohamad Hatef, Robert P. W. Duin, and Jiri Matas. On combining classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(3):226–239, 1998.

[24] Varun Kotte. PASC: Pipeline-aware conformal prediction with joint coverage guarantees for multi-stage NLP and LLM pipelines. *arXiv preprint arXiv:2605.18812*, 2026.

[25] Ahmed Lamkanfi, Serge Demeyer, Emanuel Giger, and Bart Goethals. Predicting the severity of a reported bug. In *7th IEEE Working Conference on Mining Software Repositories (MSR)*, pages 1–10, 2010.

[26] Moule Lin, Qizhen Lan, Shuhao Guan, Weipeng Jing, Jiexin Fan, David Gregg, and Goetz Botterweck. Uncertainty quantification for LLM agents: A taxonomy, an evaluation protocol, and an empirical study. *arXiv preprint arXiv:2609.07395*, 2026.

[27] Zachary C. Lipton, Yu-Xiang Wang, and Alexander J. Smola. Detecting and correcting for label shift with black box predictors. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[28] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[29] Lingsen Meng. How far can a commercial decision model's probabilities be trusted? A calibration audit of TypeSafe Jev against open and general-purpose classifiers. Code, data and paper, Zenodo, `https://doi.org/10.5281/zenodo.22935043`, 2026. Version 1.0.0.

[30] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 7076–7087, 2020.

[31] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.

[32] OpenJev. OpenJev: an open-weights decision model. `https://huggingface.co/openjev/openjev`, 2026. Hugging Face commit b4a89f5; CC BY-NC 4.0 weights; states it is independent of TypeSafe. Not the same project as other repositories named OpenJev. Accessed 2026-10-17.

[33] SangJin Park, Myungsub Choi, Jineok Kim, and Minseung Kang. Risk is not review value: Wrong-answer exposure under bounded review budgets. In *SeT-LLM Workshop at KDD*, 2026. arXiv:2609.07095.

[34] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.

[35] Qwen, An Yang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[36] Stephan Rabanser, Nathalie Rauschmayr, Achin Kulshrestha, Petra Poklukar, Wittawat Jitkrittum, Sean Augenstein, Congchao Wang, and Federico Tombari. Gatekeeper: Improving model cascades through confidence tuning. *arXiv preprint arXiv:2502.19335*, 2025.

[37] Marco Saerens, Patrice Latinne, and Christine Decaestecker. Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1):21–41, 2002.

[38] Lijuan Tang and Yuemeng Zheng. Typed decision models: An early evidence audit and evaluation checklist. *arXiv preprint arXiv:2609.32160*, 2026.

[39] Team OLMo, Pete Walsh, Luca Soldaini, et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

[40] TypeSafe. Confidence. TypeSafe documentation, `https://docs.typesafe.ai/confidence`, 2026. Accessed 2026-10-16.

[41] Xiangwei Wang, Peng Wang, and Saman Halgamuge. CORDIAL: Calibrating ordinal LLM outputs from few labels. *arXiv preprint arXiv:2609.29807*, 2026.

[42] Boming Xia, Liming Zhu, Erdun Gao, Qinghua Lu, Minhui Xue, and Dino Sejdinovic. Uncertainty propagation in LLM-based systems. *arXiv preprint arXiv:2604.23505*, 2026.

[43] Michael J. Zellinger, Rex Liu, and Matt Thomson. Cost-saving LLM cascades with early abstention. *arXiv preprint arXiv:2502.09054*, 2025.

[44] Jiaqing Zhang, Sandeep Elluri, Bhanu Cherukuvada, Yonah Joffe, Jessica Sena, Miguel Contreras, Scott Siegel, Subhash Nerella, Catherine Price, and Parisa Rashidi. Auditing multimodal LLM raters: Central tendency bias in clinical ordinal scoring. *arXiv preprint arXiv:2605.16386*, 2026.

[45] Jiaxin Zhang, Caiming Xiong, and Chien-Sheng Wu. Agentic confidence calibration. *arXiv preprint arXiv:2601.15778*, 2026.

[46] William Zhang, Saurabh Amin, and Georgia Perakis. Decomposition-based modular conformal prediction for two-stage modeling. *arXiv preprint arXiv:2510.04406*, 2025.

A Action rule and cost matrix

Table 8: The action rule $F(c_2, c_3)$. *Unclear* is checked first.

$c_2 \setminus c_3$	0	1	2
1 (yes)	accept	defer	escalate
0 (no)	accept	accept	escalate
unclear	request evidence (any c_3)		

Table 9: Frozen cost matrix $\text{cost}(a, a^*)$. One unit of human cost is added per escalation. "req. ev." = request evidence.

action \ target	accept	escalate	req. ev.	defer
accept	0	10	6	8
escalate	1	0	1	1
req. ev.	1	1	0	1
defer	1	1	1	0

B Reproducibility

Every analysed Jev call used `jev-1.13.0`. Real-data calls: Mozilla 2,424 (including a label-wording arm) plus a 252-call smoke test; kernel 2,160 plus a 216-call smoke test; Kubernetes 9,738 (1,082 issues, three nodes, three repeats; the 270-call smoke test's issues were reused), with model version pinned after 9 smoke calls had resolved to `jev-1.14.0`; those calls and one version probe are kept and excluded. All analysed calls returned HTTP 200 and parsed. The four comparison models made 3,246 deterministic passes each on the Kubernetes issues, with no failures. Every dataset, raw response, execution manifest and analysis is hash-locked in the accompanying repository, as are the pre-registration documents and their revision history. The figure data in this paper are generated from committed result files by `make_figure_data.py`. An audit on 2026-10-23 found one defect: the held-out analysis seeded its per-tracker bootstrap with Python's salted string hash, so the HR2 one-sided lower bounds varied between runs (0.640–0.644 for the kernel, 0.759–0.763 for Mozilla); point estimates and test outcomes never changed. The seed is now deterministic (bounds 0.762 and 0.648), and every other analysis, re-run under a different hash seed, was byte-identical.

C Data provenance and release

Sources. Bug reports come from the public Mozilla Bugzilla and Linux kernel Bugzilla, fetched through their REST interfaces. Texts were cleaned, privacy-scrubbed (names and e-mail addresses removed) and truncated to 2,000 characters; exclusion rules are applied by the dataset builders and recorded in manifests with SHA-256 hashes.

What is released. Bug identifiers, tracker URLs, reference labels, text hashes, every model output, every protocol with its commit history, all analysis code and every result file. Report texts are not redistributed: the builders regenerate them from the public trackers, and the hashes verify the regeneration. Bug-tracker links used in the split-integrity check are released as identifiers only.

What needs access. Re-running model calls needs a TypeSafe API key for Jev and a GPU for the open models (OpenJev at Hugging Face commit `b4a89f5` under vLLM 0.29.0; Qwen2.5, Phi-3.5 and OLMo-2 under `transformers`). Every table and figure in this paper can be regenerated offline from the released outputs, and a script checks every reported number against the result files.

D Claims and the evidence behind them

Table 10 lists each main claim, whether it was confirmatory or exploratory, and the test behind it.

Table 10: Each main claim, its status and its test. Confirmatory claims were specified before the data they use existed; exploratory claims were not, or reuse previously analysed bugs.

Claim	Status	Test and outcome
Decision estimator ranks failures above final-step confidence (Jev, new bugs)	Confirmatory (HR1)	One-sided paired bootstrap, Holm; supported
Decision estimator above chance on each tracker	Confirmatory (HR2)	One-sided bootstrap, Holm; supported
Same contrast for four further models	Confirmatory, three amendments (X1, X2)	8 tests, Holm; all supported
Same contrast in an independent workflow (Kubernetes, Jev)	Confirmatory (K1, K2)	One-sided paired bootstrap, Holm; supported
Same contrast on Kubernetes for four further models	Confirmatory (XK1, XK2)	8 tests, Holm; 3 of 4 supported; post-hoc competence check: both competent models supported
Label-free correction transfers to Kubernetes	Confirmatory (K3)	Inconclusive, negative estimate
Estimator adds nothing beyond the action's error rate (Jev-style)	Exploratory	Paired bootstrap CI of AUROC difference
Error rates invert under label shift; re-weighting and BBSE repair the ranking (BBSE mix estimate biased)	Exploratory	AUROC with bootstrap CI on held-out bugs; synthetic known-shift test
Action rule beats random review (Jev, OpenJev)	Confirmatory (I1)	One-sided paired bootstrap, Holm; supported
Action rule beats confidence-ranked review	Confirmatory (I2)	Not supported for any model
No 10% review policy beats no review	Descriptive, pre-registered design	Point estimates under the frozen cost table
Whether review pays depends on the cost ratio	Exploratory (post hoc, planned)	Frozen rankings re-scored on a cost grid
Case-level score not viable	Pre-registered feasibility check	Simulation on development data; stop rule applied
Primitive confidence profiles, threshold transfer, severity compression	Exploratory / descriptive	Development bugs, bootstrap and Wilson intervals